

14강

머신러닝

최신 딥러닝

통계·데이터과학과 박준희



✧ 학습목차

- 1 PyTorch 소개
- 2 Transformer 기본 원리
- 3 Transformer 파이썬 구현



1

PyTorch 소개



1 PyTorch 소개

● PyTorch : 인기 딥러닝 프레임워크

- 직관적이고 간결한 코드
- 풍부한 기능 및 확장성
- 강력한 커뮤니티 및 지원

현재 PyTorch는 파이썬 버전 3.9~3.12만을 지원

- 1강을 참고하여 파이썬 3.12.8을 추가로 설치
- 프로젝트 디렉토리에 dl이라는 새 가상환경 생성:
C:\machine_learning>py -3.12 -m venv dl
- 기본 패키지들 및 파이토치 설치 (pip install torch)

2

Transformer

기본 원리



2 Transformer 기본 원리

● Transformer - Vaswani et al. (2017) “Attention Is All You Need”

- Attention 메커니즘 기반 인코더-디코더 모델
- 기존 RNN/LSTM과 달리 병렬 처리(Self attention) 가능
- 긴 시퀀스에서도 장거리 의존성 효율적 학습 가능

- **Encoder**

- 입력 시퀀스를 **Self-Attention**과 **Feed-Forward Network**를 통해 임베딩 벡터로 변환한다.

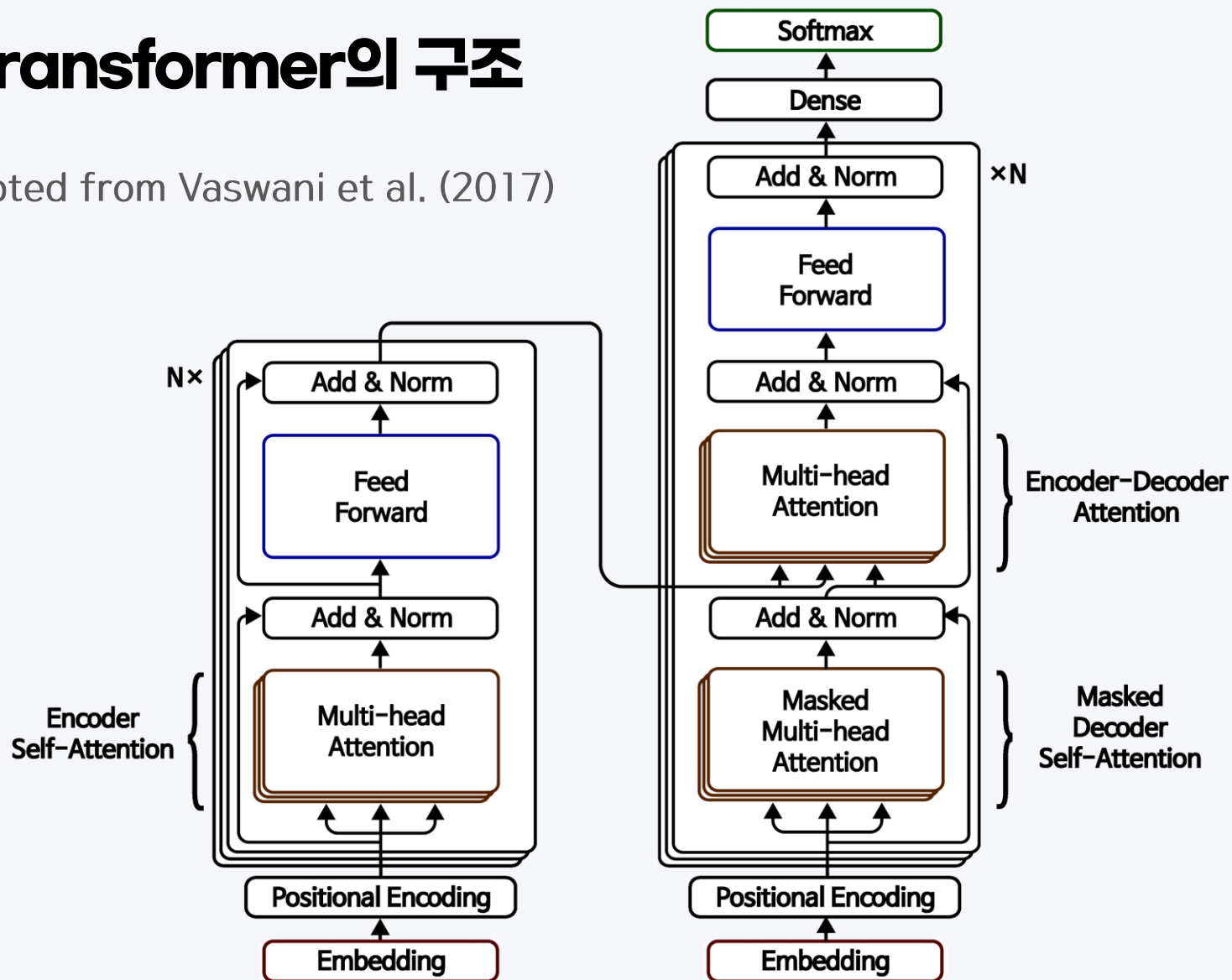
- **Decoder**

- **Masked Self-Attention**으로 미래 정보를 가리는 동시에, **Encoder-Decoder Attention**을 통해 인코더가 만들어놓은 정보를 참조하여 최종 출력을 생성한다.

2 Transformer 기본 원리

● Transformer의 구조

Adapted from Vaswani et al. (2017)



2 Transformer 기본 원리

● 핵심 구성 요소

- 임베딩 (Embedding)
- 포지셔널 인코딩 (Positional Encoding)
- 멀티헤드 어텐션 (Multi-Head Attention)
- 위치별 피드포워드 네트워크 (Position-wise FFN)
- 잔차 연결 + 레이어 정규화 (Residual + LayerNorm)

2 Transformer 기본 원리

● 인코더 (Encoder)

- 입력 임베딩 + 포지셔널 인코딩
- Self-Attention
- Feed-Forward Network(FFN)
- Residual & Layer Normalization
- 여러 계층(`n_enc_layers`) 반복

절차 예시 (개념적 코드)

```
def encoder(enc_inp):  
    # 인코더 입력 (x_enc) 준비  
    x_enc = embedding(enc_inp) * sqrt(d_model)  
    x_enc = x_enc + pos_encoding(x_enc)  
    x_enc = dropout(x_enc)
```

2 Transformer 기본 원리

● 인코더 (Encoder)

```
# 인코더 레이어 반복
for _ in range(n_enc_layers):
    # 1) Multi-Head Self-Attention
    attn_enc = multi_head_attention(x_enc, x_enc, x_enc, mask=None)
    attn_enc = dropout(attn_enc)
    attn_enc = layer_normalization(x_enc + attn_enc)

    # 2) Feed-Forward
    ff_enc = pointwise_ff(attn_enc)
    x_enc = layer_normalization(attn_enc + ff_enc)

enc_out = x_enc
return enc_out
```

2 Transformer 기본 원리

● 디코더 (Decoder)

- 입력 임베딩 + 포지셔널 인코딩
- Masked Self-Attention (미래 토큰을 가리는 **causal mask**)
- Encoder-Decoder Attention (인코더 출력 참조)
- Feed-Forward Network(FFN)
- Residual & Layer Normalization
- 여러 계층(`n_dec_layers`) 반복
- 출력(Dense & Softmax)

절차 예시 (개념적 코드)

```
def decoder(dec_inp):  
    x_dec = embedding(dec_inp) * sqrt(d_model)  
    x_dec = x_dec + pos_encoding(x_dec)  
    x_dec = dropout(x_dec)  
    mask = causal_mask(x_dec)
```

출처:파이썬 프로그램 실습내용 중

2 Transformer 기본 원리

● 디코더 (Decoder)

```
for _ in range(n_dec_layers):  
    # 1) Masked Self-Attention  
    attn1 = multi_head_attention(x_dec, x_dec, x_dec, mask)  
    attn1 = layer_normalization(attn1 + x_dec)  
  
    # 2) Encoder-Decoder Attention  
    attn2 = multi_head_attention(attn1, enc_out, enc_out, None)  
    attn2 = dropout(attn2)  
    attn2 = layer_normalization(attn1 + attn2)  
  
    # 3) Feed-Forward  
    ff_dec = pointwise_ff(attn2)  
    x_dec = layer_normalization(attn2 + ff_dec)  
  
logits = dense(x_dec)  
probs = softmax(logits, dim=-1)  
return probs
```

출처:파이썬 프로그램 실습내용 중

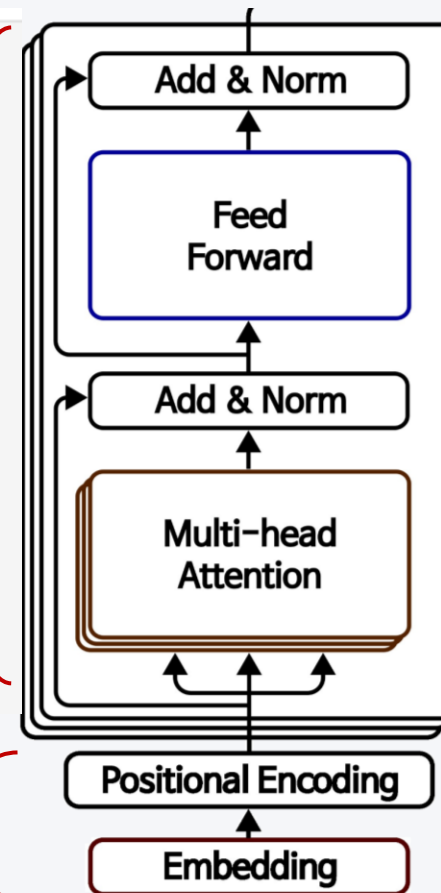
2 Transformer 기본 원리

● 전체 코드 흐름 (개념)

```
def vanilla_transformer(enc_inp, dec_inp, n_enc_layers, n_dec_layers, d_model):
    # ===== ENCODER =====
    x_enc = embedding(enc_inp) * sqrt(d_model)
    x_enc = x_enc + pos_encoding(x_enc)
    x_enc = dropout(x_enc)

    for _ in range(n_enc_layers):
        # Self-Attention + FFN
        attn_enc = multi_head_attention(x_enc, x_enc, x_enc, mask=None)
        attn_enc = dropout(attn_enc)
        attn_enc = layer_normalization(x_enc + attn_enc)

        ff_enc = pointwise_ff(attn_enc)
        x_enc = layer_normalization(attn_enc + ff_enc)
    enc_out = x_enc
```



2 Transformer 기본 원리

● 전체 코드 흐름 (개념)

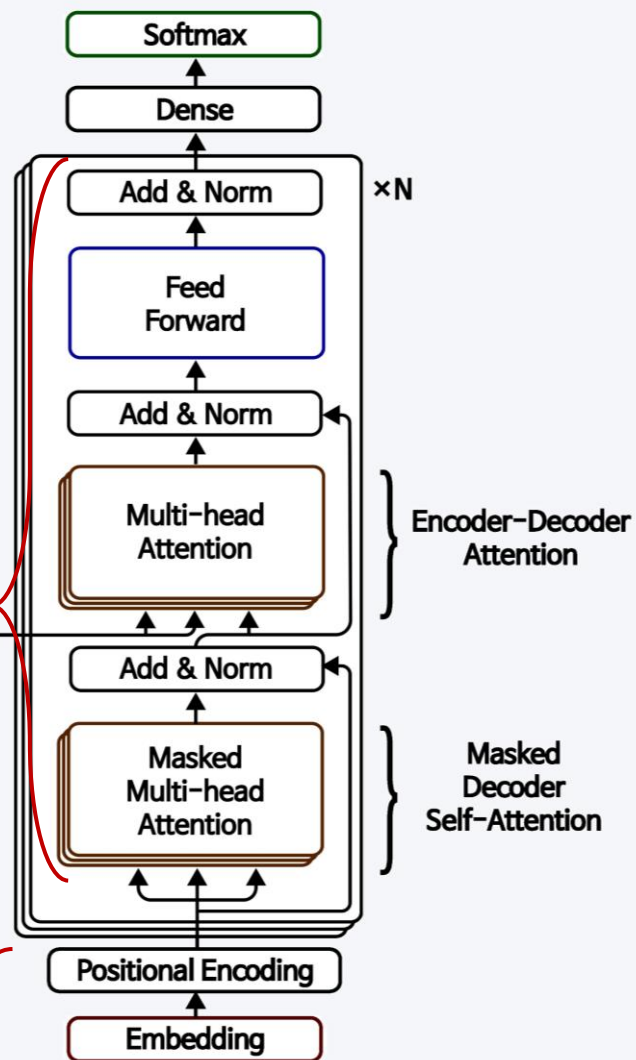
```
# ===== DECODER =====
x_dec = embedding(dec_inp) * sqrt(d_model)
x_dec = x_dec + pos_encoding(x_dec)
x_dec = dropout(x_dec)

for _ in range(n_dec_layers):
    # Masked Self-Attention
    attn1 = multi_head_attention(x_dec, x_dec, x_dec, mask=causal_mask(x_dec))
    attn1 = layer_normalization(attn1 + x_dec)

    # Encoder-Decoder Attention
    attn2 = multi_head_attention(attn1, enc_out, enc_out, mask=None)
    attn2 = dropout(attn2)
    attn2 = layer_normalization(attn1 + attn2)

    # Feed-Forward
    ff_dec = pointwise_ff(attn2)
    x_dec = layer_normalization(attn2 + ff_dec)

logits = dense(x_dec)
probs = softmax(logits, dim=-1)
return probs
```



3

Transformer

파이썬 구현



3 Transformer 파이썬 구현

● 라이브러리 импорт

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
```


3 Transformer 파이썬 구현

● 데이터 전처리

```
# 예제 문장 (독일어 -> 영어 번역)
sentences = ['das ist ein buch P', # 인코더 입력 (P: padding)
             'S that is a book',   # 디코더 입력 (S: start)
             'that is a book E']   # 디코더 목표 출력 (E: end)

# 단어 사전
src_vocab = {'P': 0, 'das': 1, 'ist': 2, 'ein': 3, 'buch': 4}
tgt_vocab = {'P': 0, 'that': 1, 'is': 2, 'a': 3, 'book': 4, 'S': 5, 'E': 6}
# 디코더 결과를 다시 단어로 매핑하기 위한 역 사전
number_dict = {i: w for i, w in enumerate(tgt_vocab)}

src_vocab_size = len(src_vocab) # 5
tgt_vocab_size = len(tgt_vocab) # 7
```

3 Transformer 파이썬 구현

● 하이퍼파라미터 설정

```
# Transformer 하이퍼파라미터 설정
src_len = 5 # 입력 문장 길이
tgt_len = 5 # 출력 문장 길이
d_model = 64 # 일반적인 경우에는 512
n_heads = 2 # 일반적인 경우에는 8
d_ff = 2048 # FC 노드 갯수 (논문대로 2048)
n_layers = 2 # 일반적인 경우에는 6
```

3 Transformer 파이썬 구현

● Batch 생성 함수

```
def make_batch(sentences):  
    input_batch = [[src_vocab[n] for n in sentences[0].split()]]  
    output_batch = [[tgt_vocab[n] for n in sentences[1].split()]]  
    target_batch = [[tgt_vocab[n] for n in sentences[2].split()]]  
    return (torch.LongTensor(input_batch),  
            torch.LongTensor(output_batch),  
            torch.LongTensor(target_batch))
```

```
enc_inputs, dec_inputs, target_batch = make_batch(sentences)  
print('Encoder Inputs:', enc_inputs)  
print('Decoder Inputs:', dec_inputs)  
print('Target Batch:', target_batch)
```

```
Encoder Inputs: tensor([[1, 2, 3, 4, 0]])  
Decoder Inputs: tensor([[5, 1, 2, 3, 4]])  
Target Batch: tensor([[1, 2, 3, 4, 6]])
```

3 Transformer 파이썬 구현

● Positional Encoding

```
def get_sinusoid_encoding_table(n_position, d_model):
    def cal_angle(position, hid_idx):
        return position / np.power(10000, 2*(hid_idx//2)/d_model)
    def get_posi_angle_vec(position):
        return [cal_angle(position, hid_j) for hid_j in range(d_model)]

    sinusoid_table = np.array([get_posi_angle_vec(pos_i) for pos_i in range(n_position)])
    sinusoid_table[:, 0::2] = np.sin(sinusoid_table[:, 0::2]) # 짝수
    sinusoid_table[:, 1::2] = np.cos(sinusoid_table[:, 1::2]) # 홀수
    return torch.FloatTensor(sinusoid_table)

pos_encoding = get_sinusoid_encoding_table(src_len + 1, d_model)
print('Positional Encoding Shape:', pos_encoding.shape)
```

$$\frac{\text{position}}{10000^{2 \times (\text{hid_idx} // 2) / d_model}}$$

Positional Encoding Shape: torch.Size([6, 64])

3 Transformer 파이썬 구현

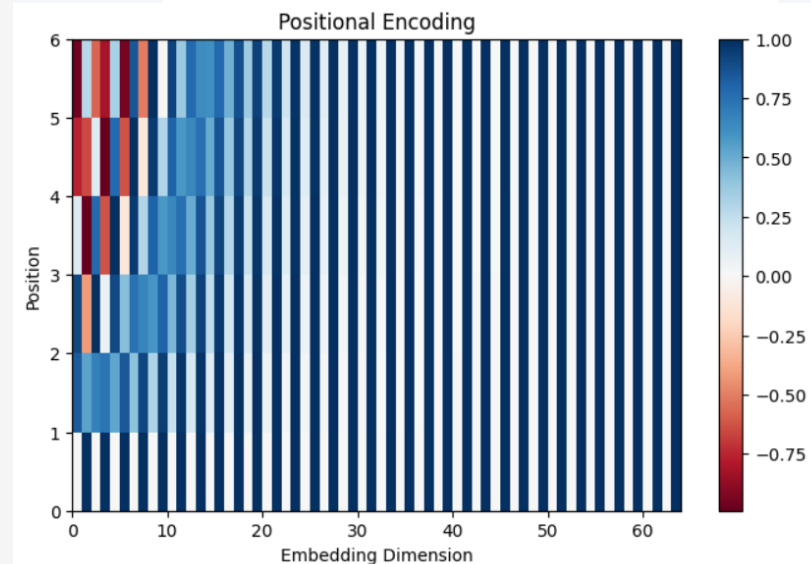
● Positional Encoding 시각화 예제

- 가로축 (Embedding Dimension): 임베딩 벡터의 차원(d_{model})
- 세로축 (Position): 시퀀스에서의 위치 인덱스(n_{position})

```
def plot_positional_encoding(pos_enc, d_model, n_position):
    plt.figure(figsize=(8, 5))
    plt.pcolormesh(pos_enc.numpy(), cmap='RdBu') # 2D 컬러맵
    plt.xlabel('Embedding Dimension')
    plt.xlim((0, d_model))
    plt.ylabel('Position')
    plt.ylim((0, n_position))
    plt.colorbar()
    plt.title("Positional Encoding")
    plt.show()
```

```
# pos_encoding: (src_len+1, d_model)
plot_positional_encoding(pos_encoding, d_model, src_len + 1)
```

$$\frac{\text{position}}{10000^{2 \times (\text{hid_idx} / 2) / d_{\text{model}}}}$$



3 Transformer 파이썬 구현

● 어텐션 마스크 생성

- 마스크1 : 어텐션 연산에서 PAD 토큰 무시

```
def get_attn_pad_mask(seq_q, seq_k):  
    # seq_q와 seq_k에서 PAD(=0) 토큰 위치를 마스크  
    batch_size, len_q = seq_q.size()  
    batch_size, len_k = seq_k.size()  
    pad_attn_mask = seq_k.data.eq(0).unsqueeze(1) # (B, 1, T_k)  
    return pad_attn_mask.expand(batch_size, len_q, len_k) # (B, T_q, T_k)
```

3 Transformer 파이썬 구현

● 어텐션 마스크 생성

- 마스크2 : 어텐션 연산에서 미래 정보 접근 막음

```
def get_attn_subsequent_mask(seq):  
    # 디코더용: 현재 시점 이후 위치를 마스크  
    attn_shape = [seq.size(1), seq.size(1)] # (T, T)  
    subsequent_mask = np.triu(np.ones(attn_shape), k=1).astype(bool)  
    return torch.from_numpy(subsequent_mask) # (T, T)
```

3 Transformer 파이썬 구현

● Encoder 레이어

```
class EncoderLayer(nn.Module):
    def __init__(self):
        super(EncoderLayer, self).__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.norm1 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model)
        )
        self.norm2 = nn.LayerNorm(d_model)
```


3 Transformer 파이썬 구현

● Encoder 레이어

```
def forward(self, x, self_attn_mask):  
    if self_attn_mask is not None:  
        B, T, S = self_attn_mask.shape # 예) (1, 5, 5)  
        self_attn_mask = self_attn_mask.repeat(n_heads, 1, 1)  
    # Self-Attention + Residual/LayerNorm  
    attn_output, attn_weights = self.self_attn(x, x, x, attn_mask=self_attn_mask)  
    x = self.norm1(x + attn_output)  
  
    ffn_output = self.ffn(x)  
    x = self.norm2(x + ffn_output)  
    return x, attn_weights
```

3 Transformer 파이썬 구현

● Decoder 레이어

```
class DecoderLayer(nn.Module):
    def __init__(self):
        super(DecoderLayer, self).__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.norm1 = nn.LayerNorm(d_model)
        self.enc_attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.norm2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model)
        )
        self.norm3 = nn.LayerNorm(d_model)
```

3 Transformer 파이썬 구현

● Decoder 레이어

```
def forward(self, dec_x, enc_output, self_attn_mask, enc_attn_mask):  
    # ---- (1) 디코더 Self-Attention 마스크 (B, T, T) -> (B*n_heads, T, T) ----  
    if self_attn_mask is not None:  
        B, T, S = self_attn_mask.shape  
        self_attn_mask = self_attn_mask.repeat(n_heads, 1, 1)  
  
    # ---- (2) 인코더-디코더 어텐션 마스크 (B, T, S) -> (B*n_heads, T, S) ----  
    if enc_attn_mask is not None:  
        B2, T2, S2 = enc_attn_mask.shape  
        enc_attn_mask = enc_attn_mask.repeat(n_heads, 1, 1)
```

3 Transformer 파이썬 구현

● Decoder 레이어

```
# 디코더 Self-Attention
```

```
self_attn_output, self_attn = self.self_attn(dec_x, dec_x, dec_x, attn_mask=self_attn_mask)  
dec_x = self.norm1(dec_x + self_attn_output)
```

```
# 인코더-디코더 Attention
```

```
enc_attn_output, enc_attn = self.enc_attn(dec_x, enc_output, enc_output, attn_mask=enc_attn_mask)  
dec_x = self.norm2(dec_x + enc_attn_output)
```

```
# Feed Forward
```

```
ffn_output = self.ffn(dec_x)  
dec_x = self.norm3(dec_x + ffn_output)  
return dec_x, self_attn, enc_attn
```

3 Transformer 파이썬 구현

● Encoder

```
class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.src_emb = nn.Embedding(src_vocab_size, d_model)
        self.pos_emb = nn.Embedding.from_pretrained(
            get_sinusoid_encoding_table(src_len + 1, d_model),
            freeze=True
        )
        self.layers = nn.ModuleList([EncoderLayer() for _ in range(n_layers)])
```

3 Transformer 파이썬 구현

● Encoder

```
def forward(self, enc_inputs):  
    # 위치 임베딩  
    positions = torch.arange(enc_inputs.size(1), device=enc_inputs.device) \  
        .unsqueeze(0).expand(enc_inputs.size(0), -1) + 1  
    output = self.src_emb(enc_inputs) + self.pos_emb(positions)  
  
    # (B, T, T)  
    enc_self_attn_mask = get_attn_pad_mask(enc_inputs, enc_inputs)  
  
    attn_weights_list = []  
    for layer in self.layers:  
        output, attn_weights = layer(output, enc_self_attn_mask)  
        attn_weights_list.append(attn_weights)  
  
    return output, attn_weights_list
```

출처:파이썬 프로그램 실습내용 중

3 Transformer 파이썬 구현

● Decoder

```
class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.tgt_emb = nn.Embedding(tgt_vocab_size, d_model)
        self.pos_emb = nn.Embedding.from_pretrained(
            get_sinusoid_encoding_table(tgt_len + 1, d_model),
            freeze=True
        )
        self.layers = nn.ModuleList([DecoderLayer() for _ in range(n_layers)])
```

3 Transformer 파이썬 구현

● Decoder

```
def forward(self, dec_inputs, enc_inputs, enc_outputs):  
    # 위치 임베딩  
    positions = torch.arange(dec_inputs.size(1), device=dec_inputs.device) \  
        .unsqueeze(0).expand(dec_inputs.size(0), -1) + 1  
    dec_output = self.tgt_emb(dec_inputs) + self.pos_emb(positions)  
  
    # 디코더 마스크  
    dec_self_attn_pad_mask = get_attn_pad_mask(dec_inputs, dec_inputs)  
    dec_self_attn_sub_mask = get_attn_subsequent_mask(dec_inputs).to(dec_inputs.device)  
    dec_self_attn_sub_mask = dec_self_attn_sub_mask.unsqueeze(0) # (1, T, T)  
    dec_self_attn_mask = dec_self_attn_pad_mask | dec_self_attn_sub_mask # (B, T, T)
```


3 Transformer 파이썬 구현

● Decoder

```
# 인코더-디코더 마스크
dec_enc_attn_mask = get_attn_pad_mask(dec_inputs, enc_inputs) # (B, T, src_len)

dec_self_attns, dec_enc_attns = [], []
for layer in self.layers:
    dec_output, self_attn, enc_attn = layer(
        dec_output,
        enc_outputs,
        dec_self_attn_mask,
        dec_enc_attn_mask
    )
    dec_self_attns.append(self_attn)
    dec_enc_attns.append(enc_attn)

return dec_output, dec_self_attns, dec_enc_attns
```

출처:파이썬 프로그램 실습내용 중

3 Transformer 파이썬 구현

● Transformer 전체 모델 구성

```
class Transformer(nn.Module):
    def __init__(self):
        super(Transformer, self).__init__()
        self.encoder = Encoder()
        self.decoder = Decoder()
        self.projection = nn.Linear(d_model, tgt_vocab_size, bias=False)

    def forward(self, enc_inputs, dec_inputs):
        enc_outputs, enc_self_attns = self.encoder(enc_inputs)
        dec_outputs, dec_self_attns, dec_enc_attns = self.decoder(dec_inputs, enc_inputs, enc_outputs)
        logits = self.projection(dec_outputs)
        # (batch_size, T, vocab_size) -> (batch_size*T, vocab_size)
        return logits.view(-1, logits.size(-1)), enc_self_attns, dec_self_attns, dec_enc_attns
```

3 Transformer 파이썬 구현

● 모델 학습

- CrossEntropyLoss, Adam 옵티마이저 사용하여 학습

```
model = Transformer()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(20):
    optimizer.zero_grad()
    enc_inputs, dec_inputs, target_batch = make_batch(sentences)

    outputs, enc_self_attns, dec_self_attns, dec_enc_attns = model(enc_inputs, dec_inputs)
    loss = criterion(outputs, target_batch.contiguous().view(-1))
    print(f"Epoch {epoch+1}, Loss = {loss.item():.6f}")
    loss.backward()
    optimizer.step()
```

3 Transformer 파이썬 구현

● 모델 테스트 및 결과 확인

```
def greedy_decoder(model, enc_input, start_symbol):
    enc_outputs, enc_self_attns = model.encoder(enc_input)
    dec_input = torch.zeros(1, tgt_len, dtype=torch.long, device=enc_input.device)
    next_symbol = start_symbol
    for i in range(tgt_len):
        dec_input[0][i] = next_symbol
        dec_outputs, _, _ = model.decoder(dec_input, enc_input, enc_outputs)
        projected = model.projection(dec_outputs)
        prob = projected.squeeze(0).max(dim=-1)[1]
        next_symbol = prob.data[i].item()
    return dec_input
```

3 Transformer 파이썬 구현

● 모델 테스트 및 결과 확인

```
enc_inputs, _, _ = make_batch(sentences)
greedy_dec_input = greedy_decoder(model, enc_inputs, start_symbol=tgt_vocab["S"])
predict, _, _, _ = model(enc_inputs, greedy_dec_input)
predict = predict.data.max(1, keepdim=True)[1]

print(sentences[0], '->', [number_dict[n.item()] for n in predict.squeeze()])

# 예상 출력 (학습 성능에 따라 달라질 수 있음):
# das ist ein buch P -> ['that', 'is', 'a', 'book', 'E']

das ist ein buch P -> ['that', 'is', 'a', 'book', 'E']
```

✦ 정리하기

- Transformer는 2017년 Vaswani의 “Attention Is All You Need” 논문에서 처음 제안된 인코더-디코더 모델
- 임베딩 : 입력 토큰을 특정 차원의 벡터로 변환
포지셔널 인코딩 : 순서 정보 부여
- Self-Attention 메커니즘 : 각 토큰이 문맥 내 다른 토큰들과 어떤 상호작용을 가지는지 효율적으로 학습
- Masked Self-Attention : 디코더가 미래 토큰을 미리 보지 않도록 차단
- 인코더 및 디코더 블록 구성 : Attention, Feed-Forward Network, Add & Norm

마무리

머신러닝

감사합니다

